KAVACH (कवच): Code & Working Prototype Guide

Architecture Map, One-Command Verification & Live Demonstration Guide

Academic Course: CSE3101 — Agentic AI (Credits: 3, LDP: 2-0-2)
Faculty Mentors: Dr. Soharab Hossain Shaikh & Mr. Pranshu Tiwari
Institution: BML Munjal University, Gurugram
Team Members: Dhruv Jain, Hemant Goyal, Rajat Yadav
Repository: `https://github.com/jaindhruv1923/KAVACH`
Deliverable Type: Mid-Term Evaluation (Phase-2) Deliverable 4

---

1. Codebase Architecture & File Mapping

Every feature presented in the midsem documentation is backed by concrete, runnable Python source files:

```
KAVACH-main/
├── 01_KAVACH_Platform/
│   ├── backend/
│   │   ├── app/
│   │   │   ├── main.py                  <-- 120 REST routes, SSE streams, middleware
│   │   │   ├── agent/
│   │   │   │   ├── orchestrator.py      <-- 10-stage FSM workflow engine (CO1)
│   │   │   │   ├── react_engine.py      <-- Bounded ReAct iterative tool loop (CO2)
│   │   │   │   ├── tools.py             <-- 9 registered sandboxed tools (CO3)
│   │   │   │   └── state.py             <-- Transactional workflow session states
│   │   │   ├── security/
│   │   │   │   ├── detector.py          <-- Context-aware PII regex scanner (CO2)
│   │   │   │   ├── secret_detector.py   <-- Shannon entropy H > 4.5 detector (CO2)
│   │   │   │   ├── token_vault.py       <-- Zero-Knowledge DPDP Act UUID vault (CO2)
│   │   │   │   ├── package_firewall.py  <-- PyPI slopsquatting real-time gate (CO2)
│   │   │   │   └── merkle_ledger.py     <-- SHA3-512 append-only audit DAG
│   │   │   ├── rag/
│   │   │   │   ├── ingest.py            <-- AST syntax chunker (classes/functions) (CO1)
│   │   │   │   └── embed_store.py       <-- Qdrant vector store (MiniLM 384-d)
│   │   │   ├── impact/
│   │   │   │   └── analyzer.py          <-- Python AST blast-radius graph G=(V,E) (CO1, CO3)
│   │   │   ├── generation/
│   │   │   │   ├── generator.py         <-- Evidence-grounded synthesis (CO2)
│   │   │   │   └── llm_client.py        <-- Dual-engine router (Gemini / Ollama)
│   │   │   ├── adk/
│   │   │   │   ├── agent.py             <-- LlmAgent & lifecycle states (CO1)
│   │   │   │   ├── callbacks.py         <-- 8-hook Google ADK callbacks (CO1, CO2)
│   │   │   │   └── medical_triage.py    <-- Canonical medical triage pipeline
│   │   │   ├── crew/
│   │   │   │   ├── agents.py            <-- 5 persona-driven crew agents (CO1)
│   │   │   │   └── flows.py             <-- Stateful @start/@listen Crew Flows
│   │   │   ├── mcp/
│   │   │   │   └── server.py            <-- Anthropic MCP JSON-RPC 2.0 server (CO3)
│   │   │   └── a2a/
│   │   │       └── protocol.py          <-- Google Agent-to-Agent message bus (CO1)
│   │   └── requirements.txt
│   ├── frontend/                        <-- Mission Control dark-mode workspace
│   │   ├── index.html, app.js, style.css
│   └── tests/                           <-- Automated Pytest test suites
├── 02_Banking_and_Results_Dashboard/
│   └── implementation_banks/
│       ├── server.py                    <-- 15 Core Banking routes mounted at /bank (CO3)
│       ├── core_banking.py              <-- Ledger accounts & ACID transactions
│       ├── advanced_banking_engines.py  <-- Distributed SAGA & Time-Lock Escrow
│       └── test_banking_100_suite.py    <-- 107 passing banking tests
├── verification/                        <-- Independent verification scripts
│   ├── focused_check.py                 <-- Traversal & token containment checks
│   └── live_e2e_integration_test.py     <-- Master 11-step integration runner
└── agentic_ai_midsem/                   <-- Official Midsem submission deliverables
```

---

2. One-Command Master Startup Guide

To launch the full unified platform (DevOps Command Center + Bharat Core Banking Highway):

Windows PowerShell (Recommended):

```
# In project root (KAVACH-main):
.\run_kavach.ps1
```

- Automatically checks dependencies, detects port collisions, initializes databases, starts FastAPI backend on `http://localhost:8000`, and opens the browser workspace.

Windows Batch / Command Prompt:

```
run_kavach.bat
```

Direct Python Launch (Zero-Configuration Offline Mode):

```
python -m uvicorn 01_KAVACH_Platform.backend.app.main:app --host 127.0.0.1 --port 8000 --reload
```

---

3. Fast Test Verification Commands (100% Pass Rate)

To verify the codebase in front of the evaluation panel:

```
# 1. Master Live E2E Integration Suite (11 checks in ~4 seconds):
python verification/live_e2e_integration_test.py

# 2. Focused Security Sandbox & Path Traversal Guard (5 checks in ~4 seconds):
python verification/focused_check.py

# 3. KAVACH Deterministic 100 Suite (100 tests in 0.38 seconds):
pytest 01_KAVACH_Platform/tests/test_kavach_100_suite.py -q

# 4. Bharat Banking 100 Suite (107 tests in 1.62 seconds):
pytest 02_Banking_and_Results_Dashboard/implementation_banks/test_banking_100_suite.py -q
```

---

4. Audited 4-Step Live Demonstration Scenarios

Follow these four reproducible scenarios during the live midsem evaluation:

Scenario 1: Malicious Prompt with Indian National Identifier (Pre-LLM Halt)

- **Goal:** Demonstrate deterministic pre-execution guardrail and zero token egress.
- **Input Prompt:** `"Create an account lookup microservice using Aadhaar number 2345 6789 0123"`
- **Action:** Submit via the dashboard prompt box or speak via Groq Whisper microphone.
- **Expected Result:** Pre-execution Sentinel immediately detects Aadhaar pattern via Verhoeff checksum. Risk engine scores 0.85 and triggers instant **`BLOCKED`** stage in **0.017 ms**. The LLM is never invoked; zero cloud tokens spent.

Scenario 2: High-Entropy Credential Exfiltration Defense

- **Goal:** Demonstrate Shannon entropy credential interception.
- **Input Code in Live Review Tab:** `python AWS_SECRET_KEY = "AKIAIOSFODNN7EXAMPLEB5F3489872134567" db_conn = "postgres://admin:SuperSecretPass123!@prod-db.internal:5432/main"`
- **Expected Result:** Shannon entropy scanner calculates H = 4.82 > 4.5. Returns immediate `BLOCKED` policy verdict with structured redaction guidance.

Scenario 3: Legitimate Safe DevOps Request (Full Grounded Lifecycle)

- **Goal:** Demonstrate full FSM lifecycle, Qdrant RAG retrieval, and AST blast radius.
- **Input Prompt:** `"Add a rate-limiting middleware to prevent brute force attacks on /auth/login"`
- **Expected Result:** 1. Sentinel clears input → **`ALLOW`**. 2. Qdrant retrieves relevant authentication context from repository. 3. AST engine analyzes dependency graph and calculates blast radius across callers. 4. Generator synthesizes evidence-grounded Python middleware. 5. AST validator parses code syntax with `ast.parse()`. 6. Final status: **`COMPLETE`** with full cryptographic audit trace displayed.

Scenario 4: Bharat Core Banking Highway & Time-Lock Escrow (`/bank`)

- **Goal:** Demonstrate mission-critical financial transaction governance and RBI compliance.
- **Action:** Navigate to `http://localhost:8000/bank`.
- **Execution:** Initiate an inter-bank transfer of **₹75,000.00** from Account `SBIN00012345678` to `ICIC00045678901`.
- **Expected Result:** Because amount exceeds the statutory **₹50,000.00 threshold**, the system automatically quarantines the transaction in **Time-Lock Smart Escrow**, triggers RBI Maker-Checker dual control, and prevents unilateral agent disbursement.